

8-Bit Precision 6T SRAM Compute-in-Memory Macro Using Global Bitline-Combining Scheme for Edge AI Chips

Jian-Wei Su[✉], Pei-Jung Lu, Ping-Chun Wu, *Graduate Student Member, IEEE*, Yen-Chi Chou, Ta-Wei Liu, Yen-Lin Chung, Li-Yang Hung, Jin-Sheng Ren, Wei-Hsing Huang, Chih-Han Chien, Peng-I Mei, Sih-Han Li[✉], Shyh-Shyuan Sheu, Wei-Chung Lo, *Member, IEEE*, Shih-Chieh Chang, Hao-Chiao Hong[✉], *Senior Member, IEEE*, Chung-Chuan Lo, Ren-Shuo Liu[✉], *Member, IEEE*, Chih-Cheng Hsieh[✉], *Senior Member, IEEE*, Kea-Tiong Tang[✉], and Meng-Fan Chang[✉], *Fellow, IEEE*

Abstract—Efforts to advance the use of analog SRAM compute-in-memory (SRAM-CIM) macros for high-precision multiply-and-accumulate (MAC) operations must deal with issues pertaining to energy efficiency, computing latency (T_{AC}), and area overhead. This brief presents a novel SRAM-CIM structure that utilizes (1) a high input precision computing cell (HIPCC) to perform 8b-MAC operations with high multiplication throughput, and (2) a global bitline-combining (GBL-comb) scheme to improve energy efficiency by reducing the number of analog-to-digital converters (ADCs). A 28nm 384-kb SRAM-CIM macro with 20-bit output precision (near-full precision) was fabricated using a foundry-provided 28nm logic process for MAC operations with 8b-input, 8b-weight, and 16 accumulations. The resulting macro achieved a T_{AC} of 3.6 ns with energy efficiency of 14.97 TOPS/W when applied to 8-bit MAC operations.

Index Terms—Artificial intelligence (AI), inference, computing-in-memory (CIM), local computing cell (LCC), static random-access memory (SRAM).

Manuscript received 23 June 2023; revised 14 October 2023; accepted 5 November 2023. Date of publication 9 November 2023; date of current version 27 March 2024. This work was supported in part by the Industrial Technology Research Institute (ITRI); in part by the Taiwan Semiconductor Research Institute (TSRI); and in part by the Ministry of Science and Technology (MOST), Taiwan. This brief was recommended by Associate Editor J. Yang. (*Corresponding author: Meng-Fan Chang.*)

Jian-Wei Su is with the Electronic and Optoelectronic System Research Laboratories (EOSL), Industrial Technology Research Institute (ITRI), Hsinchu 300044, Taiwan, and also with the Department of Electrical Engineering, National Tsing Hua University, Hsinchu 300044, Taiwan (e-mail: jwsu@gapp.nthu.edu.tw).

Pei-Jung Lu, Ping-Chun Wu, and Li-Yang Hung are with the Institute of Electronic Engineering, National Tsing Hua University, Hsinchu 300044, Taiwan.

Yen-Chi Chou, Ta-Wei Liu, Yen-Lin Chung, Jin-Sheng Ren, Wei-Hsing Huang, Chih-Han Chien, Ren-Shuo Liu, Chih-Cheng Hsieh, Kea-Tiong Tang, and Meng-Fan Chang are with the Department of Electrical Engineering, National Tsing Hua University, Hsinchu 300044, Taiwan (e-mail: mfchang@ee.nthu.edu.tw).

Peng-I Mei, Sih-Han Li, Shyh-Shyuan Sheu, Wei-Chung Lo, and Shih-Chieh Chang are with the Electronic and Optoelectronic System Research Laboratories (EOSL), Industrial Technology Research Institute (ITRI), Hsinchu 300044, Taiwan.

Hao-Chiao Hong is with the Department of Electrical and Computer Engineering, National Yang Ming Chiao Tung University, Hsinchu 30010, Taiwan.

Chung-Chuan Lo is with the Institute of Systems Neuroscience, National Tsing Hua University, Hsinchu 300044, Taiwan.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSII.2023.3331375>.

Digital Object Identifier 10.1109/TCSII.2023.3331375

1549-7747 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Beijing Normal University. Downloaded on September 08, 2026 at 07:19:04 UTC from IEEE Xplore. Restrictions apply.

I. INTRODUCTION

UNDER the conventional Von Neumann computing architecture, discrete memory and processing units are connected via a memory bus with limited data transfer capacity. This structure imposes computing latency as well as speed discrepancies between the memory and processor. Computing-in-memory (CIM) reduces the need to move data via the memory bus by enabling parallel computing within the memory array. SRAM-based memory devices have been employed to enhance the efficiency of mid-to-high precision computation [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17] and researchers have employed multi-phase computing to achieve high input and weight precision; however, that approach tends to result in long access times [14], [15], [16], [17]. ADCs have also been applied to output operations [10], [11], [16], [17]; however, that approach is not energy efficient, due to the need to transfer the ADC output to the readout circuit.

The remainder of this brief is organized as follows. Section II outlines the inherent challenges associated with multibit SRAM-CIM macros and describes the structure of the proposed high-precision SRAM-CIM macro. Section III describes the proposed high input precision computing cell used to perform in-array MAC operations with high tolerance for process variation. Section IV describes the proposed global bitline-combining (GBL-comb) scheme used to improve signal margin and achieve high energy efficiency. Section V presents simulated and measured results. Section VI concludes this brief.

II. CHALLENGES IN SRAM-CIM MACRO AND THE STRUCTURE OF THE PROPOSED SRAM-CIM MACRO

A. Long Computing Latency for Multi-Bit MAC Operations

Advanced neural networks and complex datasets require high-precision MAC operations to achieve high inference accuracy; however, high input precision increases computing latency when using the conventional bitwise [14], [15], [16], [17] input methodology with multiple input phases (see Fig. 1). It is also possible to preserve inference accuracy by increasing output precision; however, this requires a high-resolution analog-to-digital converter (ADC) with multiple output phases. Long computing latency is a serious issue when dealing with high input precision or high output precision.

B. Structure of Proposed SRAM-CIM Macro

Fig. 2 illustrates the structure of the proposed 384-kb SRAM-CIM macro with support for MAC operations

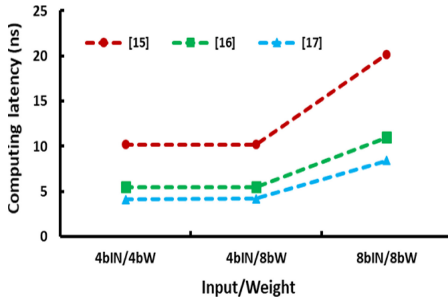


Fig. 1. Challenges in the development of SRAM-based CIM macros.

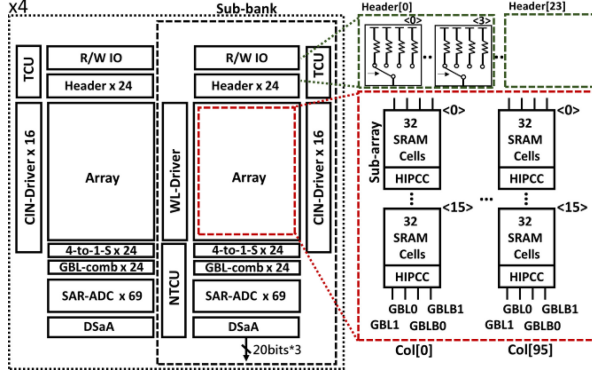


Fig. 2. Structure of proposed SRAM-CIM macro.

involving 16 accumulations of 8-bit input and 8-bit weight with 20-bit total output precision. The proposed macro includes 8 sub-banks, each of which includes a 48-kb memory cell array (1.5K sub-arrays), 16 channel-input drivers (CIN-Driver), 3 output channels, 1 normal timing control unit (NTCU), 1 CIM timing control unit (TCU), and 1 set of digital shifter-and-adder (DSaA). Each output channel includes 24 4-to-1 selectors (4-to-1-S), 24 bitline headers, 24 global bitline combining schemes (GBL-comb), and 69 conventional successive-approximation-register analog-to-digital converters (SAR-ADC). Each sub-array includes 32 6T-SRAM cells and 1 high input precision computing cell (HIPCC). The 16 sub-arrays are connected via 4 global bitlines (GBL0, GBL1, GBLB0, GBLB1) in the same column.

Fig. 3 illustrates kernel mapping for i -channel accumulation in a sub-bank of the proposed SRAM-CIM macro. All 8-bit weight data are stored in 32 6T-SRAM cells on the same row, but across 8 columns in 8 different sub-arrays. The proposed cell array uses a foundry provided compact 6T SRAM cell with aggressive layout rules to suppress area overhead.

III. PROPOSED HIGH INPUT PRECISION COMPUTING CELL

A. SRAM-CIM Macro: Sub-Array Structure

Fig. 4(a) illustrates the structure and operational waveform of a sub-array with the proposed HIPCC, which is used to perform 8-bit input and 1-bit weight multiplication. Each HIPCC is shared by 32 compact-area 6T SRAM cells in a local memory column (sub-array) and connected to 32 6T SRAM cells via a local bitline pair (LBL and LBLB). Each HIPCC comprises two pass transistors (N1, N2) and four multiplication transistors (N3, N4, N5, and N6).

B. SRAM-CIM Macro: Read and Write Modes

During write operations, data is sent to GBL and GBLB to wait for programming, and the horizontal WL (HWL) is high to turn on N1 and N2 and thereby connect GBL to LBL

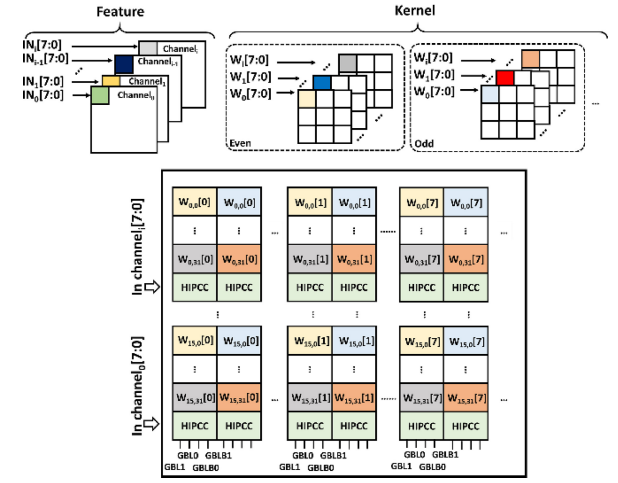
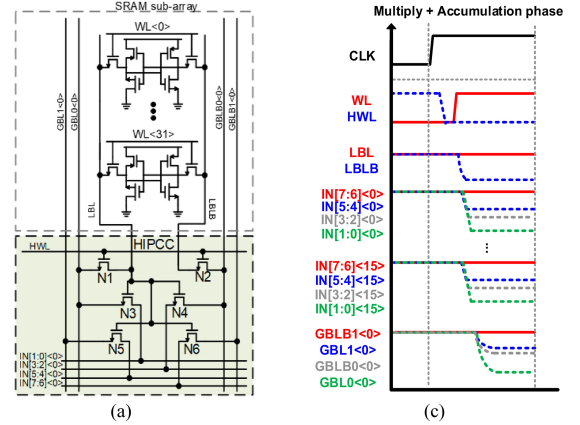


Fig. 3. MAC value kernel mapping of 16-channel accumulation operations in a memory bank.



	Multiply			One sub-array multiply results ($V_{out0}[N]$, $V_{out0}[N]$, $V_{out1}[N]$, $V_{out1}[N]$)
	1-bit W (LBL)	2-bit IN (IN[7:6], IN[5:4], IN[3:2], IN[1:0])		
HIPCC				
Value3	1	11	V_{11}	$3 \times \Delta V$
Value2	1	10	V_{10}	$2 \times \Delta V$
Value1	1	01	V_{01}	$1 \times \Delta V$
Value0	1	00	V_{00}	$0 \times \Delta V$
Value0	0	Don't care		$0 \times \Delta V$

(b)

Fig. 4. (a) Structure of HIPCC; (b) Truth table of HIPCC for computing the multiplication of 2-bit input and 1-bit weight in GBL0, GBLB0, GBL1, and GBLB1; (c) Waveform of HIPCC while computing the 2-bit input and 1-bit weight with 16 accumulations on GBL0, GBLB0, GBL1, and GBLB1.

and GBLB to LBLB. For instance, if the programmed bit is equal to 1, then the LBL is driven to VDD, while the LBLB is grounded through N1 and N2. The selected WL is then turned on to enable the writing of data to the selected SRAM cell via the LBL and LBLB.

During read operations, LBL and LBLB are respectively precharged to VDD before activating the WL by connecting LBL to GBL and LBLB to GBLB (HWL is high). The selected WL is then turned on to readout the data stored in 6T SRAM cell. For instance, if the data stored in memory bitcell is "1", then the LBL is maintained at the pre-charge voltage, while LBLB is discharged to ground. The voltage of LBL is then transmitted to GBL0 via N1, and the voltage of LBLB is transmitted to GBLB0 via N2. Finally, a sense amplifier (SA) reads out the voltage difference between GBL and GBLB to output a digital readout result.

C. SRAM-CIM Macro: Multiplication

Fig. 4(b) presents a truth table of the HIPCC during multiplication operations. A given 8-bit input is divided into 4 2-bit inputs, each of which is converted into 4-level analog voltage using a DAC to reduce computing latency. A larger 2-bit input is converted to a higher analog voltage, while a smaller 2-bit input is converted to a lower voltage. The multiplication result is shown on GBL0, GBLB0, GBL1, and GBLB1 in the voltage domain via the interaction between input voltage and N3, N4, N5, and N6. Note that GBL0, GBLB0, GBL1, and GBLB1 can each generate a 2-bit input using a 1-bit weight partial-MAC value (pMACV). Note that during multiplication, the HWL is low to turn off N1 and N2 and thereby disconnect LBL from GBL and LBLB from GBLB.

In each sub-array, a single selected wordline (WL) is activated to pass 1-bit weight data (e.g., W[0]) stored in the accessed SRAM cell to the LBL to control N3, N4, N5, and N6 in a HIPCC. If W[0] is zero, then N3, N4, N5, and N6 are off, indicating that the multiplication result is equal to 0. If W[0] is one, then N3, N4, N5, and N6 in HIPCC are turned on to pass the 2-bit analog voltage (V_{00} , V_{01} , V_{10} , V_{11}) to GBL0, GBLB0, GBL1, and GBLB1. The voltage on GBLB1 represents the multiplication result of the two most significant bits (MSBs; IN[7:6]) and 1-bit weight. The voltage on GBL1 represents the multiplication result of IN[5:4] and 1-bit weight. The voltage on GBLB0 represents the multiplication result of IN[3:2] and 1-bit weight, and the voltage on GBL0 represents the multiplication result of the two least significant bits (LSBs; IN[1:0]) and 1-bit weight.

D. Operational Details of SRAM-CIM Macro: Accumulation

Accumulation is achieved by connecting GBL0, GBL1, GBLB0, and GBLB1 in each HIPCC. The post-connected voltage of the GBLB1 (V_{GBLB1}) represents the pMACV of 2-bit input and 1-bit weight with 16-channel accumulation ($V_{GBLB1} = \sum_{i=0}^{15} (IN_i[7:6] \times W_i[k])$), where i indicates the number of accumulation=16 and k indicates the bitwise weight (e.g., W[7], W[6], ..., W[0]). The voltage of the activated GBL1 (V_{GBL1}) represents the pMACV derived as $V_{GBL1} = \sum_{i=0}^{15} (IN_i[5:4] \times W_i[k])$. The voltage of the activated GBLB0 (V_{GBLB0}) indicates the pMACV of $V_{GBLB0} = \sum_{i=0}^{15} (IN_i[3:2] \times W_i[k])$, whereas the voltage of GBL0 (V_{GBL0}) indicates the pMACV of $V_{GBL0} = \sum_{i=0}^{15} (IN_i[1:0] \times W_i[k])$. The four voltages associated with GBL0, GBLB0, GBL1, and GBLB1 then used to perform bitline combining to enable readout operations with reduced energy consumption. The SAR-ADC then converts the analog partial sum (PS) voltage obtained from global bitline combining (GBL-comb) into a digital result representing partial MAC values (pMACV_{8bIN-1bW-16A}). It is important to note that each global bitline corresponds to a specific 8-bit weight value. For instance, consider the weight relationship between the least significant bits W[0] and W[1], which is 1:2 (e.g., GBL0[0]:GBL0[1]=1:2). Consequently, the digital shifter-and-adder shifts the digital result of GBL0[1] by one bit become n+1-bit. Finally, a shifter-and-adder is necessary to combine the eight sets of pMACV_{8bIN-1bW-16A}, ultimately generating the final 20-bit output ($DOUT = \sum_{i=0}^{15} IN_i[7:0] \times W_i[7:0]$).

IV. GLOBAL BITLINE COMBINING SCHEME

The proposed global bitline combining (GBL-comb) scheme improves energy efficiency without significantly compromising system-level inference accuracy by reducing the number of ADC.

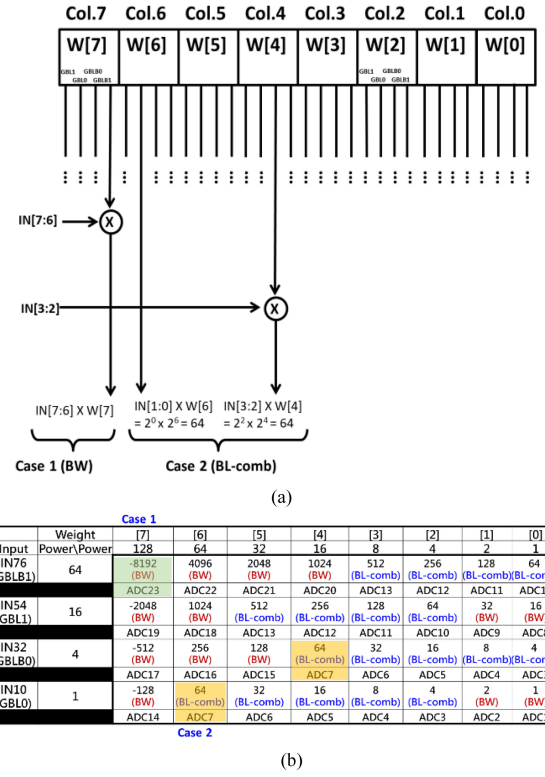


Fig. 5. (a) Implementation of proposed GBL-comb scheme in two operating modes; (b) Table of all partial sums corresponding to GBL-comb modes.

As shown in Figs. 5 (a) and (b), the GBL-comb scheme can be implemented in two modes: bit-wise (BW) and bitline combining (BL-comb). In BW mode, the pMACV representing $IN[7:6] \times W[7]$ is directly readout by a single ADC. For example, the pMACV on GBLB1 is directly readout by ADC23 without any combining.

In BL-comb mode, pMACV with the same place value are combined by the comb-circuit and read out by a single ADC. For example, $IN[1:0] \times W[6]$ and $IN[3:2] \times W[4]$ has the same place after partial MAC operations in different bitlines; therefore, they are combined by the comb-circuit to be read out by ADC7. Before incorporating the GBL-comb scheme, each sub-bank requires 96 conventional SAR-ADCs for readout operation. However, after implementing the BL-GBL combine scheme, each sub-bank requires only 69 ADCs, reducing 27 ADCs in total.

Fig. 6 (a) presents a diagram of the proposed voltage stacking circuit used to implement the proposed BL-comb scheme. Voltage stacking is achieved via capacitive coupling, which involved combining two different voltages (pMACVs) into a post-summation voltage representing MACV. BL-comb with voltage stacking, which involves two capacitors ($C1$ and $C2$) and five switches ($S[1:5]$). The capacitance ratio of $C1$ and $C2$ is 1:1, and all capacitors are fabricated using the Metal-Oxide-Metal technique. The top plate of $C1$ is connected to the partial sum node (PS) and the bottom plate of $C1$ is connected to node-1 (N1). The top plate of $C2$ is connected to N1 and the bottom plate of $C2$ is connected to VSS. Switch-1 ($S[1]$) is connected between V_{GBL-A} (GBL0 or GBLB0 or GBL1 or GBLB1) and PS, while switch-2 ($S[2]$) is connected between VSS and PS.

Switch-3 ($S[3]$) is connected between V_{GBL-B} (GBL0 or GBLB0 or GBL1 or GBLB1) and N1. Switch-4 ($S[4]$) is connected between reference voltage (V_{Ref-H}) and N1, while switch-5 ($S[5]$) is connected between VSS and N1.

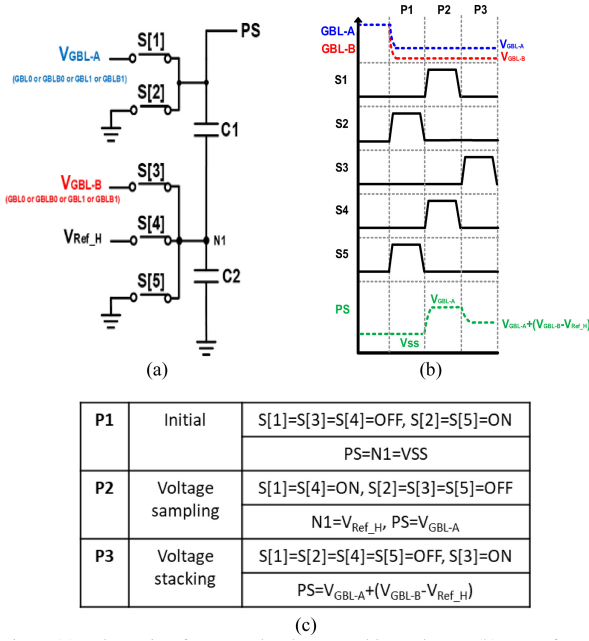


Fig. 6. (a) Schematic of proposed voltage stacking scheme; (b) Waveform of voltage stacking circuit without compensation capacitor; (c) Operation table.

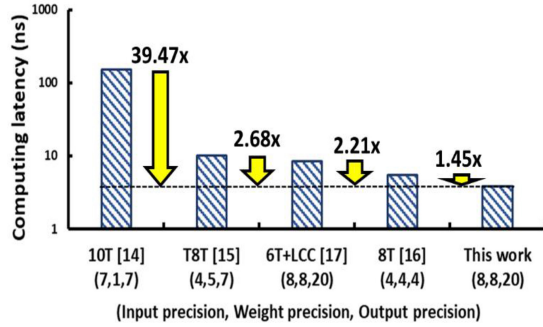


Fig. 7. Comparison of proposed SRAM-CIM macro with recent silicon verified SRAM-CIM works in terms of measured computing latency.

Figs. 6 (b) and (c) present the operational waveform and an operation table of the voltage stacking circuit through three operational phases: 1) initial phase; 2) voltage sampling phase; and 3) voltage stacking phase.

During phase-1 (initial), S[1], S[3], and S[4] are turned off, while S[2] and S[5] are turned on, such that PS and N1 are VSS.

During phase-2 (Voltage sampling), S[1] and S[4] are turned on while S[2], S[3], S[5] are turned off, such that the voltage at N1 is V_{Ref_H} and PS is V_{GBL-A} . At the same time, C1 samples the input voltage (V_{GBL-A}) from GBL and the reference voltage (V_{Ref_H}) from the reference cell array, which has a MAC value of 0.

During phase-3 (voltage stacking), S[1], S[2], S[4], S[5] are turned off and S[3] is turned on, such that the voltage of PS is $V_{GBL-A} + (V_{GBL-B} - V_{Ref_H})$ via AC coupling through C1.

V. PERFORMANCE AND MEASUREMENT RESULTS OF PROPOSED SCHEMES

A. Performance and Area of Proposed Schemes

Fig. 7 compares the proposed SRAM-CIM macro with recent silicon-verified SRAM-CIM works in terms of measured computing latency [14], [15], [16], [17]. The proposed HIPCC was shown to reduce computing latency

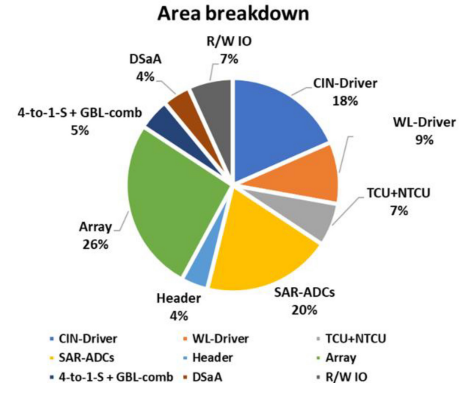


Fig. 8. Area breakdown of proposed 384-kb SRAM CIM macro.

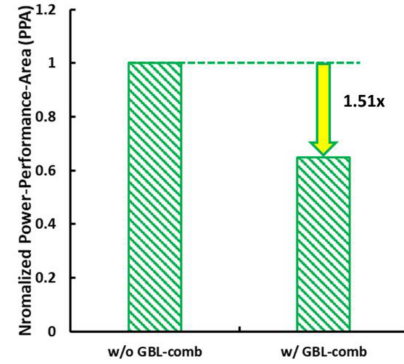


Fig. 9. Normalized power-performance-area (PPA) of a 384-kb SRAM CIM macro with and without GBL-comb.

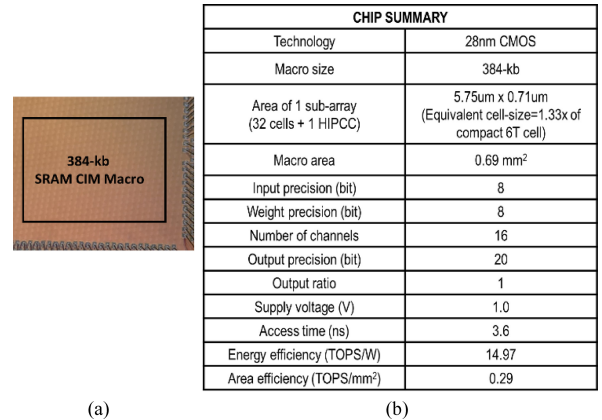


Fig. 10. (a) Die photograph and (b) chip summary table.

(T_{AC}) by 1.45x to 39.47x when performing 8-bit input and 8-bit weight MAC operations.

Fig. 8 presents an area breakdown of the proposed 384-kb SRAM CIM macro. The array consumed 26% of the macro area, the SAR-ADCs consumed 20%, the CIN-Driver consumed 18%, and the WL-Driver consumed 9%.

B. Measurement Results

Fig. 9 presents the measured power-performance-area (PPA) of the proposed 384-kb SRAM-CIM macro with and without the GBL-comb scheme. By reducing the number of ADCs used for reading out pMACV on the various bitlines, the GBL-comb scheme allowed a 1.51x reduction in PPA at the macro level.

Fig. 10 (a) presents a die photograph of the proposed SRAM-CIM macro and Fig. 10 (b) presents a summary table

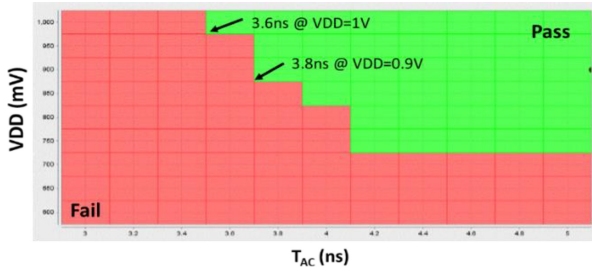


Fig. 11. Measured shmoo plot showing VDD versus T_{AC} during inference stage under 8-bit precision.

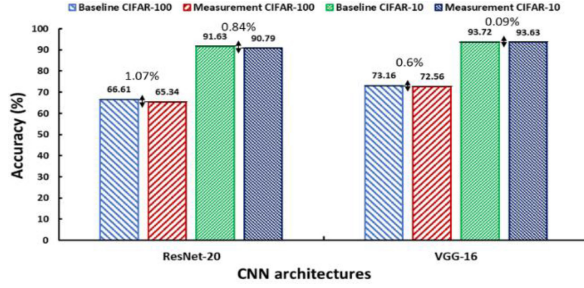


Fig. 12. Measured inference accuracy of proposed SRAM-CIM macro using ResNet-20 and VGG-16 models trained for CIFAR-10 and CIFAR-100 datasets.

TABLE I
COMPARISON TABLE OF PROPOSED AND EXISTING
SILICON-VERIFIED SRAM CIM MACROS

	[14]	[18]	[16]	[19]	This work
Array size	256x64 (16Kb)	64x60	4Kb	64Kb	384Kb
Process	65nm	55nm	7nm	28nm	28nm
Cell structure	1T1	8T1	RDBT	6T	6T+H1PCC
Input precision	7	4	4	8	8
Weight precision	1	5	4	8	8
Output precision	4	7	4	20	20
Output channel	256	12	16	8	24
Access time (ns)	150	10.2	5.5	8.4	3.5
Model	CNN	CNN	CNN	CNN	CNN
GOPS	10.7	21.2	372.4	30.48	202.1
Energy efficiency (TOPS/W)	28.1	18.37	351	11.54	14.97
Area Efficiency (TOPS/mm ²)	0.16	0.57	116.38	0.11	0.29

for convolutional neural network (CNN) operations under 8-bit input, 8-bit weight, and 20-bit output precision. The proposed SRAM-CIM macro achieved 14.97 TOPS/W during CNN inference stage with near full output ratio, where the output ratio is defined as the actual output precision divided by the ideal output precision.

Fig. 11 presents a measured shmoo plot from the 384-kb SRAM-CIM macro performing MAC operations involving 16 accumulations of 8-bit input and 8-bit weight with 20-bit output. These results confirm that the proposed SRAM-CIM macro achieved T_{AC} of 3.8ns ($V_{DD} = 0.9$ V) and 3.6ns ($V_{DD} = 1$ V).

Fig. 12 presents the measured system-level inference accuracy of the proposed SRAM-CIM test chip when applying the ResNet-20 and VGG-16 models to the CIFAR-10 and CIFAR-100 datasets. Using the ResNet-20 model, the inference accuracy was 0.84% and 1.07% below that of software simulation when respectively applied to the CIFAR-10 and CIFAR-100 datasets. Using the VGG-16 model, the inference accuracy was 0.09% (CIFAR-10) and 0.6% (CIFAR-100) below that of software simulation.

Table I compares the performance of our proposed CIM macro with that of recently silicon-verified SRAM CIM macros. The proposed CIM macro achieved the shortest computing access time while maintaining high precision in input, weight, and output.

VI. CONCLUSION

This brief presents a 384-kb SRAM-CIM macro that uses a high input precision computing cell to reduce energy consumption and T_{AC} in MAC operations. In experiments, a fabricated 28nm 384-kb SRAM-CIM macro achieved a T_{AC} of 3.6ns and energy efficiency of 14.97 TOPS/W for MAC operations with 8-bit input, 8-bit weight, and 20-bit output.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude for their collaborators at NTHU-TSMC JDP, MOST-Taiwan, TSRI and ITRI.

REFERENCES

- [1] H. Valavi, P. J. Ramadge, E. Nestler, and N. Verma, "A mixed-signal binarized convolutional-neural-network accelerator integrating dense weight storage and multiplication for reduced data movement," in *IEEE Symp. VLSI Circuits Dig. Tech. Papers*, 2018, pp. 141–142.
- [2] Z. Jiang, S. Yin, M. Seok, and J.-S. Seo, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," in *Proc. IEEE Symp. VLSI Technol.*, 2018, pp. 173–174.
- [3] S. Okumura, M. Yabuuchi, K. Hijioka, and K. Nose, "A ternary based bit scalable, 8.80 TOPS/W CNN accelerator with many-core processing-in-memory architecture with 896K synapses/mm²," in *Proc. IEEE Symp. VLSI Circuits*, 2019, pp. C248–C249.
- [4] J. Kim et al., "Area-efficient and variation-tolerant in-memory BNN computing using 6T SRAM array," in *Proc. IEEE Symp. VLSI Circuits*, 2019, pp. C118–C119.
- [5] Z. Zhang et al., "A 55nm 1-to-8 bit configurable 6T SRAM based computing-in-memory unit-macro for CNN-based AI edge processors," in *Proc. IEEE Asian Solid-State Circuits Conf. (ASSCC)*, 2019, pp. 217–218.
- [6] R. Guo et al., "A 5.1pJ/Neuron 127.3us/Inference RNN-based speech recognition processor using 16 computing-in-memory SRAM macros in 65nm CMOS," in *Proc. IEEE Symp. VLSI Circuits*, 2019, pp. C120–C121.
- [7] J. Wang et al., "A compute SRAM with bit-serial integer/floating-point operations for programmable in-memory vector acceleration," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Paper (ISSCC)*, 2019, pp. 224–226.
- [8] J. Yang et al., "Sandwich-RAM: An energy-efficient in-memory BWN architecture with pulse-width modulation," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2019, pp. 394–395.
- [9] C.-X. Xue et al., "A 22nm 2Mb ReRAM compute-in-memory macro with 121-28TOPS/W for multibit MAC computing for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2020, pp. 244–246.
- [10] J.-W. Su et al., "28nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2020, pp. 240–242.
- [11] J.-W. Su et al., "16.3 A 28nm 384kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2021, pp. 250–252.
- [12] J. Yue et al., "15.2 A 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2021, pp. 238–240.
- [13] P.-C. Wu et al., "11.8 A 28nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2022, pp. 190–192.
- [14] A. Biswas and A. P. Chandrakasan, "Conv-RAM: An energy-efficient SRAM with embedded convolution computation for low-power CNN-based machine learning applications," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2018, pp. 488–489.
- [15] X. Si et al., "A twin-8T SRAM computation-in-memory macro for multiple-bits CNN-based machine learning," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2019, pp. 396–397.
- [16] Q. Dong et al., "A 351TOPS/W and 372.4GOPS compute-in-memory SRAM macro in 7nm FinFET CMOS for machine-learning applications," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2020, pp. 242–244.
- [17] X. Si et al., "A 28nm 64Kb 6T SRAM computing-in-memory macro with 8-bit MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2020, pp. 246–248.
- [18] X. Si et al., "A twin-8T SRAM computation-in-memory unit-macro for multibit CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, Jan. 2020.
- [19] X. Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep. 2021.